

EXTRAS TEHNIC DIN LUCRAREA DE LICENȚĂ

Cum este construită tehnic aplicația TrainingIT

Explicație tehnică structurată după paginile 26-48 din newfile4

Arhitectură • straturi • date • flux de înscriere • design patterns • limite reale

Material pentru prezentarea la susținerea lucrării de licență

1. Ideea tehnică centrală

Sursa tehnică în newfile4: p. 26-28 și p. 46-47

De reținut: TrainingIT este un singur produs funcțional, dar este construit din două aplicații autonome: un frontend Next.js și un backend Java/Spring Boot. Ele nu împart cod, sesiune sau componente interne; comunică exclusiv prin HTTP și mesaje JSON.

Frontendul gestionează interfața, navigarea și starea locală a utilizatorului. Backendul implementează regulile de afaceri, accesul la date și reacțiile administrative. MariaDB păstrează atât informațiile educaționale, cât și evidența CRM. Această separare permite modificarea interfeței fără rescrierea domeniului Java și modificarea backendului fără recompilarea frontendului, atât timp cât este păstrat contractul REST/JSON.

Traseul unei cereri

- 1. Frontend** Next.js / React colectează acțiunea utilizatorului
- 2. REST/JSON** lib/api.ts trimite cererea HTTP către backend
- 3. Controller** deserializare, validare web și delegare; fără logică de afaceri
- 4. CrmFacade** punct unic de intrare și coordonare a domeniului
- 5. Servicii / comenzi** aplică regulile aplicației și modifică starea
- 6. Repository / DAO** transformă operația de domeniu în SQL explicit
- 7. JDBC / HikariCP / MariaDB** execută și persistă datele
- 8. EventBus** ramifică reacțiile secundare: audit, notificări și statistici

2. Tehnologiile care alcătuiesc sistemul

Sursa tehnică în newfile4: p. 26, p. 32-37 și p. 46

Zonă	Tehnologie	Rol în aplicație
Interfață	Next.js 16, React 19, TypeScript 5, Tailwind CSS v4	SPA autonomă, App Router, componente și interfața ambelor portaluri
Comunicare	HTTP + REST/JSON	Singurul contract dintre frontend și backend
Timp real	Server-Sent Events (SSE)	Actualizarea statisticilor publice fără polling și fără refresh
Backend web	Spring Boot 3.5	Expune controllere REST și integrează nucleul Java
Domeniu	Java + design patterns	Reguli de afaceri, servicii, comenzi, evenimente și observatori

Zonă	Tehnologie	Rol în aplicație
Persistență	MariaDB, JDBC, HikariCP	SQL explicit, pool de conexiuni și schemă controlată de aplicație
AI opțional	Anthropic Claude	Chatbot, recomandări, asistență și traducere dinamică
Documente	Apache POI și OpenPDF	Import/export Excel și generare de documente PDF

Alegere arhitecturală: Spring Boot este folosit în principal ca strat web. Nucleul de domeniu Java își gestionează singur configurația, conexiunile, schema și tranzacțiile, ceea ce reduce dependența logicii de afaceri față de framework.

3. Cum este construit backendul

Sursa tehnică în newfile4: p. 26-32 și p. 41-45

Stratul web: Aproximativ șaisprezece controllere REST din `crm.web.controller` transformă cererile HTTP în apeluri de domeniu și returnează DTO-uri. Ele sunt intenționate subțiri.

Facade: Toate operațiile importante intră prin `CrmFacade`. Controllerul nu trebuie să cunoască serviciile, comenzile sau repository-urile care vor fi activate.

Serviciile: Componentele `crm.service.*` conțin regulile pentru contacte, cursuri, înscrieri, facturare, analiză, recenzii și suport. Serviciile centrale sunt configurate ca Singleton.

Comenzile: Operațiile care modifică starea pot fi încapsulate prin Command și executate prin `CommandInvoker`, păstrând clară responsabilitatea fiecărei modificări.

Persistența: Repository-urile oferă operații orientate spre domeniu; DAO-urile execută `PreparedStatement`-uri și SQL nativ prin JDBC. Nu sunt folosite JPA sau Hibernate.

Evenimentele: După o modificare importantă, serviciul publică un eveniment. `EventBus` notifică observatori independenți, astfel încât operația principală nu cunoaște toate reacțiile secundare.

Contractul unitar de eroare

`GlobalExceptionHandler`, implementat cu `@RestControllerAdvice`, transformă excepțiile domeniului într-un răspuns `ApiError` cu status, mesaj, cale și detalii:

- resursă inexistentă → HTTP 404 Not Found;
- validare eșuată → HTTP 400 Bad Request;
- regulă de afaceri încălcată → HTTP 422 Unprocessable Entity;
- eroare de persistență → HTTP 500 Internal Server Error;

- serviciul AI indisponibil → HTTP 503 Service Unavailable.

Formulare orală: Controllerul primește cererea, CrmFacade găsește traseul, serviciul aplică regula, iar repository-ul și DAO-ul salvează rezultatul.

4. Cum este construit frontendul

Sursa tehnică în newfile4: p. 32-37

Aplicație autonomă: Frontendul se află într-un director separat, rulează independent și tratează backendul ca pe un serviciu extern accesibil numai prin REST/JSON.

App Router: Fiecare director din src/app definește o rută. Sunt separate paginile publice, autentificarea, zonele utilizatorului și ramura administrativă /admin/*.

Root layout: app/layout.tsx furnizează cadrul comun: header, navigație, selector de limbă, temă, footer, chatbot, raportarea problemelor, TranslationProvider și AuthGuard.

Gateway API: lib/api.ts centralizează get/post/put/patch/delete/upload, adresa backendului, Content-Type, cache: no-store și conversia erorilor în ApiError.

Contract tipizat: lib/types.ts oglindește DTO-urile serverului prin interfețe TypeScript precum PublicCourse, PublicStats, MyPurchase și AuthSession.

Starea aplicației: useState și useEffect gestionează starea locală; localStorage păstrează sesiunea, limba și tema; evenimentele globale și React Context propagă schimbările fără Redux, MobX sau Zustand.

SSE: EventSource se conectează la /api/public/stats/stream și actualizează automat cursurile, cursanții, ratingul mediu și recenziile.

5. Persistența și modelul de date

Sursa tehnică în newfile4: p. 29-32 și p. 42-43

DatabaseConnection este un Singleton care gestionează poolul HikariCP. La pornire, WebSchemaInitializer.ensureTables() execută schema.sql, iar nucleul Java își creează și își controlează schema în MariaDB. Spring DataSource este dezactivat intenționat.

De ce fără ORM: Legăturile sunt păstrate ca identificatori Long și SQL-ul rămâne explicit. Avantajul este controlul determinist și transparența; costul este mai mult cod și gestionarea manuală a relațiilor.

Cele trei subdomenii

- Educațional: Course → CourseSession → Enrollment. Ratingul și feedbackul sunt păstrate în Enrollment.
- CRM și vânzări: Contact, Opportunity, Activity și Employee.
- Ședințe individuale și facturare: MeditationSession legată de Invoice.

Contact poate reprezenta o persoană individuală sau o companie. Clientul/cursantul nu are o entitate Learner separată, ci este modelat prin Contact. Stările și categoriile sunt controlate prin enumerări pentru rol, tip de contact, lead, oportunitate, înscriere, plată, categorie de curs, mod de livrare și stare de sesiune.

6. Cum sunt realizate funcțiile prezentate în conspect

Sursa tehnică în newfile4: p. 29-37 și p. 38-45

Catalog și cumpărare: CourseCard.tsx afișează cursul și trimite POST către /api/public/courses/{id}/purchase prin lib/api.ts.

Cursurile cumpărate: Înscrierea este persistată ca Enrollment asociat unui Contact și unei CourseSession.

Recenzii: rating și feedback sunt câmpuri ale Enrollment, ceea ce leagă evaluarea de înscriere.

CRM administrativ: ramura /admin/* oferă ecrane pentru contacte, cursuri, achiziții, facturare, analize, angajați și sesizări; backendul le servește prin același domeniu.

Ședințe individuale: entitățile MeditationSession și Invoice modelează rezervarea cu trainerul și factura asociată.

Raportare: ReportService și ReportController generează Excel/PDF, iar EmployeeExcelImporter realizează importul în masă al angajaților.

Asistent și recomandări: ClaudeClient și AiService oferă chatbot, recomandări de curs, sprijin comercial și traducere.

Actualizări live: PublicStatsBroadcaster recalculează statisticile și le transmite prin SSE numai când există un client conectat.

7. Fluxul tehnic al cumpărării și înscrierii

Sursa tehnică în newfile4: p. 38-45

De ce este fluxul reprezentativ: O singură apăsare a butonului Enroll traversează aproape toate straturile aplicației și activează majoritatea șabloanelor de proiectare.

1. CourseCard.tsx verifică dacă utilizatorul este deja înscris și afișează Enroll sau Enrolled.
2. La click, frontendul trimite prin lib/api.ts un POST cu e-mailul, prenumele și numele utilizatorului.
3. PublicCatalogController.purchase() deserializează cererea și o delegă imediat către CrmFacade.
4. CrmFacade transmite operația către ReviewService.purchaseCourse().
5. ReviewService verifică existența și starea activă a cursului.
6. findOrCreateContact() caută contactul după e-mail; dacă lipsește, îl creează în CRM.
7. Contactul nou trece prin lanțul Email → Phone → ContactType → GDPR; Strategy calculează scorul inițial, iar Repository/DAO îl persistă.
8. ContactCreatedEvent este publicat, iar WelcomeEmailObserver construiește notificarea de bun venit.
9. Serviciul verifică idempotența; dacă există deja Enrollment pentru contact și curs, returnează înregistrarea existentă.
10. resolveSession(course) selectează sesiunea, iar EnrollmentService validează contactul și sesiunea.
11. Enrollment.builder() creează înscrierea cu status CONFIRMED; EnrollmentRepository și EnrollmentDao execută INSERT în MariaDB.
12. Leadul contactului devine ENROLLED, apoi EnrollmentCreatedEvent declanșează auditul și confirmarea.
13. Răspunsul urcă prin serviciu, facade și controller; frontendul primește HTTP 201 și schimbă butonul în Enrolled.
14. PublicStatsBroadcaster detectează modificarea și transmite valorile noi prin SSE către paginile deschise.

Idempotență: Repetarea aceleiași cereri nu creează o a doua înscriere; este returnată înregistrarea existentă.

8. Șabloanele de proiectare care susțin fluxul

Sursa tehnică în newfile4: p. 27-28 și p. 38-45

Șablon	Unde apare	Ce rezolvă
Facade	CrmFacade	Punct unic de intrare în domeniu; ascunde serviciile și pașii interni.
Singleton	Servicii, AppConfig, DatabaseConnection	O singură instanță controlată pentru componentele centrale.
Command	Operații administrative și modificări de stare	Încapsulează operația ca unitate de execuție.
Repository + DAO	Accesul la MariaDB	Repository-ul vorbește în termenii domeniului; DAO-ul execută SQL/JDBC.
Builder	Construirea Enrollment	Asamblează obiectul complex și îl inițializează cu status CONFIRMED.
Observer + EventBus	ContactCreatedEvent, EnrollmentCreatedEvent	Declanșează reacții secundare fără a cupla serviciul principal de observatori.
Factory	Notificări e-mail și SMS	Construiește mesaje adaptate canalului prin aceeași interfață.
Strategy	Scorarea leadurilor B2C/B2B	Selectează algoritmul potrivit tipului de contact.
Chain of Responsibility	Validarea contactului	Parcurge succesiv e-mail, telefon, tip de contact și GDPR și acumulează erorile.

EventBus formează o ramură laterală față de traseul principal. Serviciul publică faptul că s-a produs o schimbare, iar observatorii decid independent ce reacții execută. Astfel, auditul, notificarea și recalcularea statisticilor pot fi adăugate sau modificate fără rescrierea operației de înscriere.

9. AI, traducere, temă și notificări

Sursa tehnică în newfile4: p. 32, p. 36 și p. 41-43

AI cu degradare controlată: ClaudeClient este un wrapper peste SDK-ul Anthropic. Dacă ANTHROPIC_API_KEY lipsește, isEnabled() întoarce false și este produsă o eroare controlată 503, fără oprirea aplicației.

Traducere dinamică: TranslationProvider gestionează limba. Motorul extrage textul englez, îl trimite în loturi de maximum 25 de șiruri, limitează concurența la trei apeluri și reaplică traducerile prin MutationObserver. Rezultatele sunt memorate în localStorage.

Temă: Tailwind CSS v4 definește tokenurile vizuale. Un script rulat înainte de hidratarea React aplică imediat tema memorată sau preferința sistemului, evitând afișarea temporară a temei greșite.

Notificări: NotificationService folosește fabrici separate pentru e-mail și SMS. E-mailul este canalul principal, iar SMS-ul este construit numai dacă există număr de telefon.

Limită actuală: Metoda finală send() nu transmite încă mesaje reale; scrie doar în log. Integrarea SMTP/Twilio este marcată ca dezvoltare viitoare.

10. Ce este implementat demonstrativ, nu pentru producție

Sursa tehnică în newfile4: p. 32 și p. 46-48

- Autentificarea și rolul sunt păstrate în localStorage și controlate în principal de AuthGuard și de layoutul administrativ.
- Nu există încă sesiuni reale sau JWT, parole stocate prin hashing, RBAC complet pe server ori autentificare multifactor.
- CORS este restricționat la originea serverului Next.js, dar aceasta nu înlocuiește securitatea server-side.
- Fluxul de cumpărare confirmă instant Enrollment și nu conține un checkout real; Stripe sau Netopia sunt propuneri viitoare.
- Notificările e-mail/SMS sunt construite corect ca structură, dar nu sunt conectate la furnizori externi.
- GdprValidator conține o condiție inertă, indicată în lucrare drept un silent bug ce trebuie corectat.
- Numele MeditationSession este moștenit și ar trebui schimbat în TrainerSession sau Booking.
- Executarea directă a schema.sql ar trebui înlocuită cu migrații controlate, precum Flyway sau Liquibase, iar operațiile pe mai multe tabele necesită atomicitate mai strictă.

Formulare corectă la susținere: Aplicația demonstrează o arhitectură completă end-to-end și design patterns aplicate real, dar securitatea, plățile și mesajele externe trebuie consolidate pentru producție.

11. Explicația orală, în aproximativ 90 de secunde

TrainingIT este construit din două aplicații autonome. Frontendul este o aplicație Next.js și React, iar backendul este un nucleu Java expus prin Spring Boot. Cele două comunică numai prin REST și JSON. În backend, controllerul este subțire și delegă cererea către CrmFacade. De acolo, serviciile aplică regulile de afaceri, iar repository-urile și DAO-urile persistă datele în MariaDB prin JDBC și HikariCP. Nu este folosit un ORM, pentru ca SQL-ul și traseul datelor să rămână explicite.

Partea distinctivă este nucleul bazat pe evenimente. După o acțiune importantă, cum este înscrierea la un curs, EventBus notifică observatori independenți care pot scrie în audit, pregăti confirmări și actualiza statisticile. În fluxul de cumpărare sunt folosite împreună Facade, Singleton, Repository/DAO, Builder, Strategy, Chain of Responsibility, Observer și Factory. Frontendul primește răspunsul HTTP, iar statisticile sunt actualizate în timp real prin SSE.

AI-ul Claude este opțional și deservește chatbotul, recomandările și traducerea; dacă lipsește cheia, restul aplicației continuă să funcționeze. Implementarea este completă ca demonstrație academică, însă autentificarea este încă client-side, plata este simulată, iar mesajele e-mail și SMS sunt doar înregistrate în log.

Cele 7 idei pe care trebuie să le reții

1. Două aplicații autonome, un singur produs: Next.js în față, Java/Spring Boot în spate.
2. Comunicare exclusiv prin REST/JSON; actualizări live prin SSE.
3. Controller → CrmFacade → servicii → repository → DAO/JDBC → MariaDB.
4. Fără ORM: SQL explicit și relații gestionate manual.
5. EventBus și Observer transformă o singură acțiune în mai multe reacții independente.
6. Fluxul de înscriere este idempotent și activează mai multe design patterns.
7. AI-ul este opțional; securitatea, plata și trimiterea notificărilor trebuie maturizate pentru producție.

12. Unde se găsește această parte în lucrarea mare

Pagini	Conținut tehnic relevant
26-28	Arhitectura generală și arhitectura backend
29-31	Entități, subdomenii, relații și enumerări
32	Persistență, AI, raportare și securitate

Pagini	Conținut tehnic relevant
32-37	Arhitectura frontend
38-45	Studiul de caz al cumpărării și înscrierii
46-48	Contribuții, limite și dezvoltări necesare

Concluzie: Pentru prezentarea tehnică a produsului din conspect, nucleul relevant al lucrării newfile4 este concentrat în paginile 26-48, cu studiul de caz esențial al înscrierii în paginile 38-45.